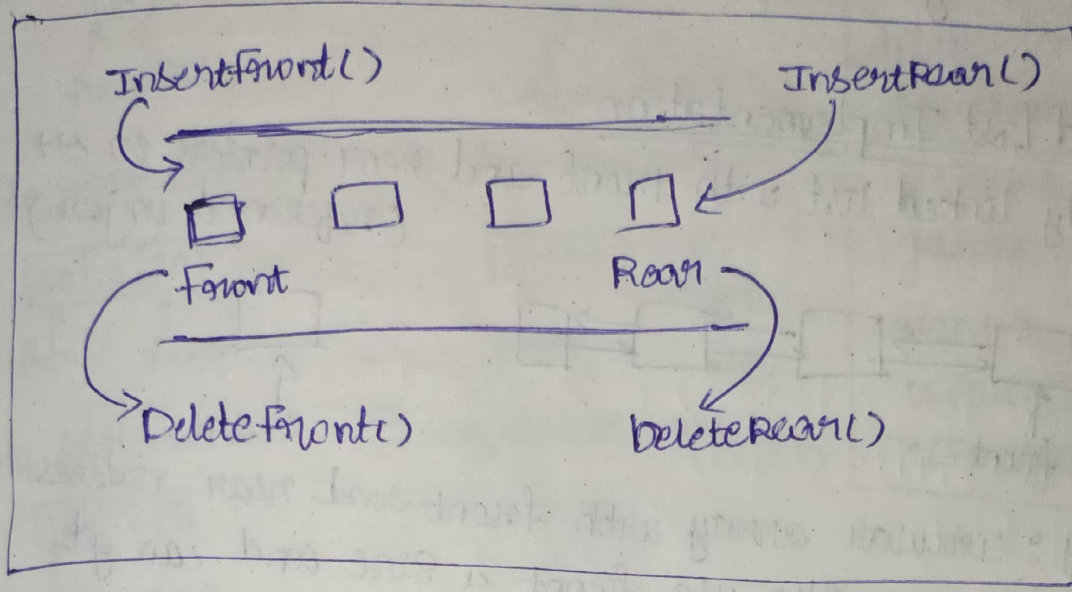


Deque data structure:

doubly ended queue



Additional operations:

getFront()

getRear()

isFull()

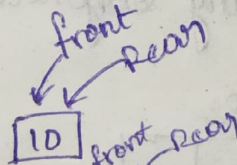
isEmpty()

size()

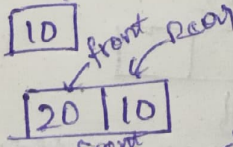
⇒



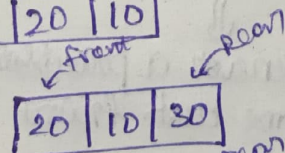
insertFront(10)



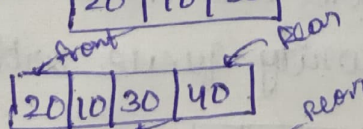
insertFront(20)



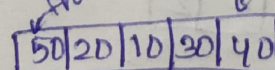
insertRear(30)



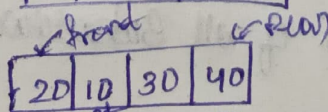
insertRear(40)



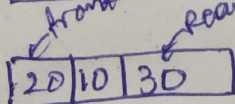
insertFront(50)



deleteFront()



deleteRear()

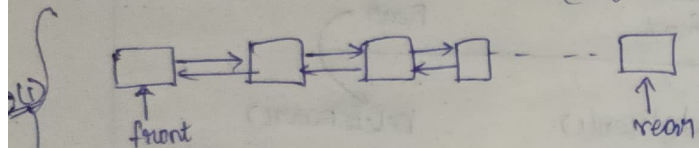


Implementation

- ↳ Linked list
- ↳ Array.

Linked List Implementation

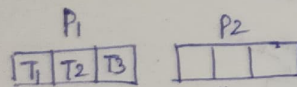
Doubly linked list with front and rear pointers in C++ (references in java)



Array: circular array with front and rear pointers. we can also use front a size and can get rear using size.

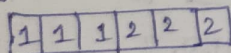
Applications:

- ① A Deque can be used as both stack and queue
- ② maintaining History of actions
- ③ A steal process scheduling algorithm



whenever a process is free, then it steal some task from rear of P1.

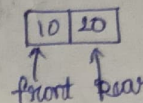
- ④ Implementing a priority queue with two types of priorities



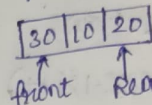
- ⑤ maximum / minimum of all subarrays of size k in an array.

Array Implementation of Deque

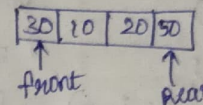
- ① dq.insertFront(10)
- ② dq.insertRear(20)



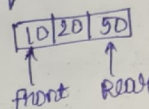
- ③ dq.insertFront(30)



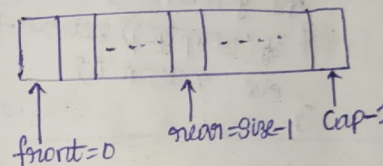
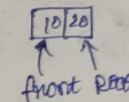
- ④ dq.insertRear(50)



- ⑤ dq.deleteFront()



- ⑥ dq.deleteRear()



struct Deque

```
{
    int size, cap;
    int *arr;
```

Deque(int c)

```
{
    cap = c;
    size = 0;
    arr = new int[cap];
```

3 - - -

```
insertFront()
insertRear()
deleteFront()
deleteRear()
getFront()
getRear()
size()
isFull()
isEmpty()
```



```
bool isEmpty() { return (size == 0); }
```

```
bool isFull() { return (size == cap); }
```

```
void insertRear(int x)
```

```
{ if (isFull()) return;
```

```
arr[size] = x;
```

```
size++;
```

```
}
```

```
void insertFront(int x)
```

```
{ if (isFull()) return;
```

```
for (int i = size-1; i >= 0; i--)
```

```
arr[i+1] = arr[i];
```

```
arr[0] = x;
```

```
size++;
```

```
}
```

```
int getRear()
```

```
{ if (isEmpty()) return -1;
```

```
else return (arr[size-1]);
```

```
}
```

```
void deleteRear()
```

```
{ if (isEmpty()) return;
```

```
size--;
```

```
}
```

```
void deleteFront()
```

```
{ if (isEmpty()) return;
```

```
for (int i = 0; i < size-1; i++)
```

```
arr[i] = arr[i+1];
```

```
size--;
```

```
}
```

```
int getFront()
```

```
{ if (isEmpty()) return -1;
```

```
else return arr[0];
```

```
}
```

struct Deque

```
{ int *arr;
```

```
int front, cap, size;
```

```
Deque (int C)
```

```
{ arr = new int[C];
```

```
cap = C;
```

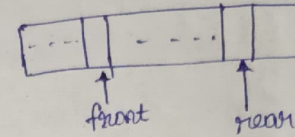
```
front = 0;
```

```
size = 0;
```

3 functions can be added *

front = 0

Rear is always $(front + size - 1) \% cap$



$\Rightarrow$ deleteFront()

$front = (front + 1) \% cap$

$\Rightarrow$ insertRear(x)

$rear = (rear + 1) \% cap$

$\Rightarrow$ insertFront(x)

$front = (front - 1 + cap) \% cap$

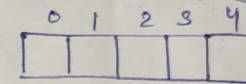
$\Rightarrow$ deleteRear()

$rear = (rear - 1 + cap) \% cap$

$rear = (front + size - 1) \% cap$

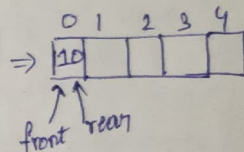
cap = 5, size = 0

front = 0



insertRear(10) $\Rightarrow$ $rear = (0 + 0 - 1) \% 5 = -1$

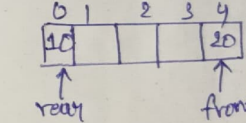
$\Rightarrow$ $rear = (-1 + 1) \% 5 = 0$



insertFront(20)

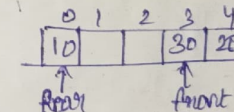
front = 4

$\Rightarrow$ $front = (0 - 1 + 5) \% 5 = 4$



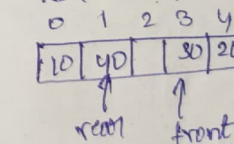
insertFront(30)

front = 3



insertRear(40)

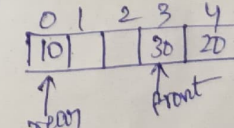
front = 3, rear = 1



deleteRear()

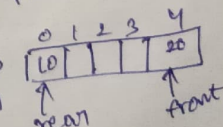
rear = 0

front = 3



deleteFront()

front = 4



efficient Implementation

```
void deleteFront()
{
    if (isEmpty()) return;
    front = (front + 1) % cap;
    size--;
}
```

```
void insertRear (int x)
{
    if (isFull()) return;
    int new_rear = (front + size) % cap;
    arr[new_rear] = x;
    size++;
}
```

```
int getFront()
{
    if (isEmpty()) return -1;
    else return 0;
}
```

```
void insertFront (int x)
{
    if (isFull()) return;
    front = (front - 1 + cap) % cap;
    arr[front] = x;
    size++;
}
```

new_rear = (front + size - 1) % cap
new_rear = (front + 1) % cap
= (front + size + 1) % cap

```
void deleteRear()
{
    if (isEmpty()) return;
    size--;
}
```

```
int getRear()
{
    if (isEmpty()) return -1;
    else return (size - 1);
    return (front + size - 1) % cap;
}
```


Deque in C++ STL:

```
#include <iostream>
```

```
#include <deque>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    deque<int> dq = {10, 20, 30};
```

```
    dq.push_front(5);
```

```
    dq.push_back(50);
```

```
    for(auto x: dq)
```

```
        cout << x << " ";
```

```
    cout << dq.front() << " " << dq.back();
```

```
    return 0;
```

```
}
```

```
int main()
```

```
{
```

```
    deque<int> dq = {10, 15, 30, 5, 12};
```

```
    auto it = dq.begin();
```

```
    it++;
```

```
    dq.insert(it, 20);
```

```
    dq.pop_front();
```

```
    dq.pop_back();
```

```
    cout << dq.size();
```

```
    return 0;
```

```
}
```

How does deque work internally?

doubly linked list provides

- push-front
- push-back
- pop-front
- pop-back

all operations in $O(1)$

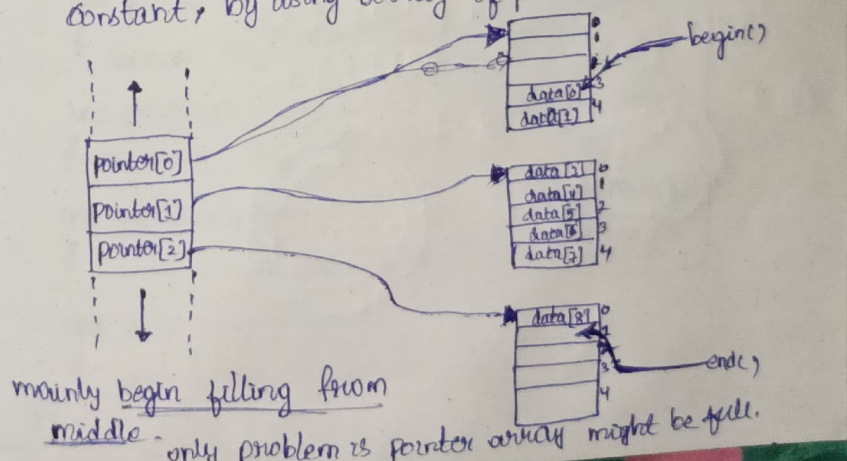
But DLL, doesn't provide random access in $O(1)$

circular array provides all operations & Random access in $O(1)$. but the problem is if the deque grows beyond the capacity, then we have to double the size and copy the data.

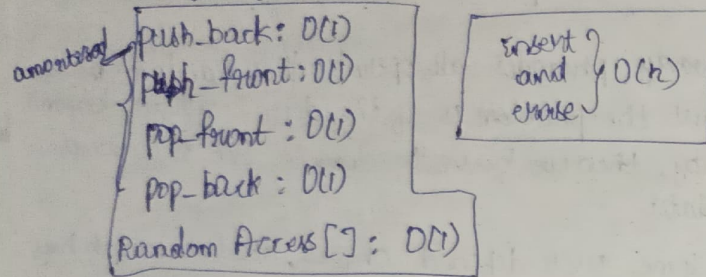
data is some user defined object, student which has name, first name, last name, many other members. So double size circular array creation is costly.

⇒ So deque uses some tricky implementation, its constant time in terms of no of copy constructors called.

⇒ So if the data is the class objects then our idea is to minimise the no of copy constructors, and no of copy constructor calls that happen is constant, by using array of pointers.



if pointer array is full, like we do in vectors doubling.
 ⇒ this particular doubling doesn't copy the whole data, it involves only copying the pointers.
 So because of that reason only constant time in terms of copy constructor calls.



The t.c of push-back & push-front is arguable, means it's not constant in worst case. you have an array of pointers that you might have to resize.

⇒ Resizing happens very rarely, because we chunks of range reqs. so the array is very unlikely to become full.

Data Structure with min/max operations

Design a data structure that supports following operations in $O(1)$ T.C.

- ① insertMin(x)
 - ② insertMax(x)
 - ③ getMin()
 - ④ getMax()
 - ⑤ extractMin()
 - ⑥ extractMax()
- get
Remove

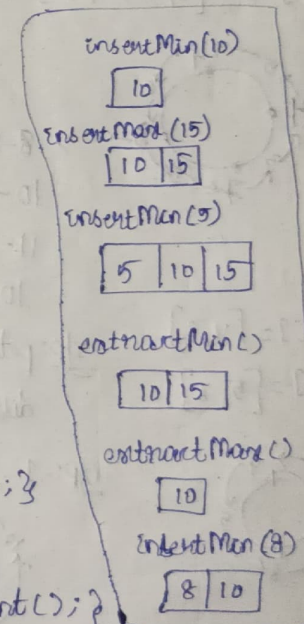
Ex: insertMin(5)
 insertMax(10)
 insertMin(3)
 insertMax(15)
 insertMin(2)
 getMin() ⇒ 2
 getMax() ⇒ 15
 insertMin(1)
 getMin() ⇒ 1
 insertMax(20)
 getMax() ⇒ 20

O/p: 2
15
1
20

struct MyDp

```

{ deque<int> dq;
void insertMin(int x)
{ dq.push-front(x); }
void insertMax(int x)
{ dq.push-back(x); }
int getMin()
{ return dq.front(); }
int getMax()
{ return dq.back(); }
int extractMin()
{ return dq.pop-front(); }
int extractMax()
{ return dq.pop-back(); }
}
    
```



first circular tour

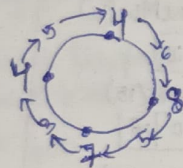
- Circular arrangement of petrol pumps
- ∞ Capacity truck
- truck consumes 1 unit of petrol for 1 unit of distance.

task: find petrol pump, such that if we begin from that pump using truck, such that we should be able to visit all petrol pumps, and we should be able to come back to this ^{initial} petrol pump.

if (exist) $\Rightarrow$ print
else $\Rightarrow$ return -1

Ex: petrol[] = [4, 8, 7, 4]
dist[] = [6, 5, 3, 5]

Q/P: 2



$$\begin{aligned} 8-5+7 &= 10 \\ 10-3+4 &= 11 \\ 11-5+4 &= 10 \\ 10-6 &= 4 \end{aligned}$$

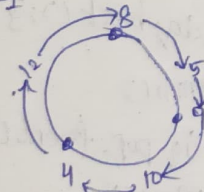
Ex: petrol[] = [4, 8]
dist[] = [5, 6]

Q/P: 2



Ex: petrol[] = [8, 9, 4]
dist[] = [5, 10, 12]

Q/P: -1



Naive solution:

$$\begin{aligned} T.C &= O(n^2) \\ A.S &= O(1) \end{aligned}$$

int firstPetrolPump (int petrol[], int dist[], int n)

```
{
    for (int start=0; start < n; start++)
    {
        int curr_petrol = 0;
        int end = start;
        while (true)
        {
            curr_petrol += (petrol[end] - dist[end]);
            if (curr_petrol < 0)
                break;
            end = (end + 1) % n;
            if (end == start)
                return (start + 1);
        }
    }
}
```

return -1;

petrol[] = [4, 8, 7, 4]

dist[] = [6, 5, 3, 5]

Start = 0: end = 0, curr_petrol = 0
curr_petrol = 0 + (4 - 6) = -2

Start = 1: end = 1, curr_petrol = 0
curr_petrol = 0 + (8 - 5) = 3, end = 2
curr_petrol = 3 + (7 - 3) = 7, end = 3
curr_petrol = 7 + (4 - 5) = 6, end = 0
curr_petrol = 6 + (4 - 6) = 4, end = 1
return 2



Better solution: (use a Deque)

- ① Keep adding items to the end of deque while $\text{curr_ptr} \geq 0$.
- ② Keep removing items from the front of deque while $\text{curr_ptr} < 0$.

petrol[j] = [50, 10, 60, 100]

$$\text{dest}[] = [80, 20, 100, 10]$$

$$T.C = O(n)$$

$$A.S = O(n)$$

deque [], cur-petrol=0

degree $\boxed{0}$, curv-petrol = $0 + (50-30) = 20$

degree $\begin{bmatrix} 0 & 1 \end{bmatrix}$, $\text{cumprod} = 20 + (10 - 20) = 10$

deque

0	1	2
---	---	---

, $curr_petrol = 10 + (60 - 100) = -30$

~~Expe~~

0	1	2	3
---	---	---	---

, ~~Grouped~~ -

degree

1	2
---	---

, $\text{curriculum} = -30 - (50 - 30) = -50$

de que $\boxed{2}$, $\text{compensat} = -50 - (10 - 20) = -40$

deque $\square$, $\text{arr_petrol} = -40 - (60 - 100) = 0$

de que $\boxed{3}$, $\text{unprofit} = 0 + (100 - 10) = 90$

deque

3	0
---	---

, $\text{arr_petrol} = 90 + (50 - 30) = 110$

deque

3	0	1
---	---	---

, curr_ptr = 110 + (10 - 20) = 100

deque

3	0	1	2
---	---	---	---

, curr_ptr = 100 + (60 - 100) = 60

Efficient solution:

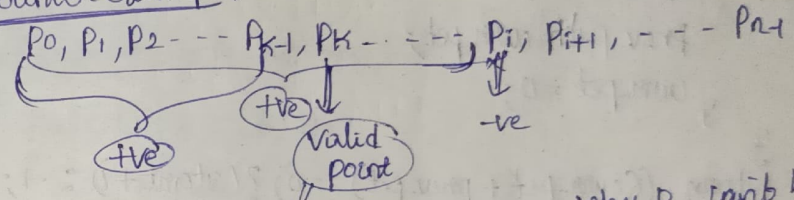
If cur_petrol becomes negative at p_i , then none of the petrol pumps from p_0 to p_i can be solution.

$$p_0, p_1, \dots, p_{i-1}, p_i, p_{i+1}, \dots, p_{n-1}$$

let P_i be the first petrol pump where current petrol becomes negative, then

$$\text{curr_petrol} = \sum_{j=0}^i (\text{petrol}[j] - \text{dist}[j])$$

Counter example:



but we have to show why P_k can't be a valid point

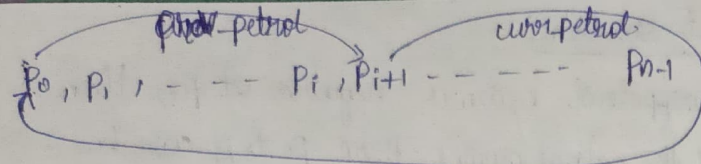
wherever we consider P_k as a valid answer,
then P_i is also considered to be as an valid answer
but here we do subtracting +ve value from
-ve value

$$P_0, P_1, P_2, \dots, P_{k-1}, P_k$$

$P_0, P_1, P_2, \dots, P_{k-1}$ +ve P_k
↓
Valid
(-ve)

$(-ve) - (+ve) = -ve$

p_i must be (-ve) if we choose p_k as a starting point.
so none of the points before p_i ~~can be~~ ^{can be} as the starting point.
so we can safely pick p_{i+1} as a starting point.



int firstPetrolPump(int petrol[], int dist[], int n)

```

{
    int start = 0, curPetrol = 0;
    int prevPet = 0;
    for (int i = 0; i < n; i++)
    {
        curPetrol += (petrol[i] - dist[i]);
        if (curPetrol < 0)
        {
            start = i + 1;
            prevPet = curPetrol;
            curPetrol = 0;
        }
    }
    return (curPetrol + prevPet) >= 0 ? (start + 1) : -1;
}

```

petrol[] = [50, 10, 60, 100]
 dist[] = [30, 20, 100, 10]

Initially: start = 0, curPet = 0, prevPet = 0

i = 0 : curPet = 0 + (50 - 30) = 20

i = 1 : curPet = 20 + (10 - 20) = 10

i = 2 : curPet = 10 + (60 - 100) = -30

start = 3

prevPet = -30

curPet = 0

i = 3 : curPet = 0 + (100 - 10) = 90

Since (curPet + prevPet) >= 0, 90 - 30 = 60
 return (start + 1) = 4 as an answer

Maximums in all subarrays of size k

Ip: arr[] = [10, 8, 5, 12, 15, 7, 6]

k = 3

(n - k + 1) subarrays

Op: 10 12 15 15 15

Ip: arr[] = [20, 5, 3, 8, 6, 15]

k = 4

Op: 20 8 15

Naive: $O(n^2)$

void printMaxK(int arr[], int n, int k)

```

{
    for (int i = 0; i < n; i++)
    {
        int max = arr[i];
        for (int j = i + 1; j < i + k; j++)
            max = max(arr[j], max);
        cout << max << " ";
    }
}

```

=> [10, 8, 5, 12, 15, 7, 6]
 i = 0 k = 3

max = 10

i = 0 : max = 10

i = 1 : max = 12

i = 2 : max = 15

i = 3 : max = 15

i = 4 : max = 15



Efficient: $O(n)$

$[0, 1, 2, 3, 4, 5, 6]$
 $[10, 8, 5, 12, 15, 7, 6]$

$k=3$

$i=0$: dq

10		
0		

$i=1$: dq

10	8	
0	1	

$i=2$: dq

10	8	5
0	1	2

 $\Rightarrow 10$

$i=3$: dq

12		
3		

 $\Rightarrow 12$

$i=4$: dq

15		
4		

 $\Rightarrow 15$

$i=5$: dq

15	7	
4	5	

 $\Rightarrow 15$

$i=6$: dq

15	7	6
4	5	6

 $\Rightarrow 15$

$[5, 8, 10]$
 dq:

5	8	
---	---	--

 dq:

8		
---	--	--

 dq:

10		
----	--	--

whenever we see an elem
 we delete all the items
 which are smaller than
 the current from the
 deque. bcs they become
 useless, they never become
 max in this window or
 subsequent windows.

when greater element,
 remove all smaller items
 and insert greater element
 at front.

when smaller element, insert
 at back.

void printmax(int arr[], int n, int k)

{ deque<int> dq;

for (int i=0; i<k; i++)

{ while(!dq.empty() && arr[i] >= arr[dq.back()]);
 dq.pop_back();

3 dq.push_back(i);

for (int i=k; i<n; i++)

{ cout << arr[dq.front()] << " ";
 while(!dq.empty() && dq.front() <= i-k)

3 dq.pop_front();
 while(!dq.empty() && arr[i] >= arr[dq.back()])
 dq.pop_back();

3 dq.push_back(i);

3 cout << arr[dq.front()] << " ";
 }